

Project on Tomographic Reconstruction

In this project you will implement a regularized reconstruction method for 3D tomography in ODL¹ and apply it to a realistic 3D CT dataset. The following instructions assume that you use **ODL version 1.0.0**. Note that the namespaces changed substantially in ODL 1.0 relative to older releases.

Problem description

Let x be some image, and let the application of the ray transform (forward) operator A to x with additive noise e result in y , i.e., $y = A(x) + e$. Your goal is to estimate x . Since this problem is ill-posed, the typical approach is to add regularization. In this project you are going to use *Huber regularization*. This is a *total variation* type of regularization, where the ℓ_1 -norm is replaced with the Huber norm H_ϵ (Huber). H_ϵ is defined in finite dimension by

$$H_\epsilon(z) = \sum_i h_\epsilon(|z_i|), \quad (1)$$

for some $\epsilon > 0$, where $h_\epsilon: \mathbb{R} \rightarrow \mathbb{R}$ is the Huber function

$$h_\epsilon(t) = \begin{cases} \frac{t^2}{2\epsilon} & \text{if } |t| \leq \epsilon, \\ |t| - \frac{\epsilon}{2} & \text{otherwise.} \end{cases} \quad (2)$$

For small ϵ , the Huber norm is a differentiable approximation of the ℓ_1 -norm. In ODL this smoothing parameter is called `gamma` and is passed positionally, i.e., `odl.functionals.Huber(space, gamma)` implements exactly the H_ϵ above with $\epsilon = \text{gamma}$. Thus, instead of solving the least-squares problem

$$\min_x \frac{1}{2} \|A(x) - y\|_2^2, \quad (3)$$

you must choose parameters $\lambda > 0$, $\epsilon > 0$, and solve the *regularized problem*

$$\min_x \frac{1}{2} \|A(x) - y\|_2^2 + \lambda H_\epsilon(\nabla x), \quad (4)$$

where $\|\cdot\|_2$ is the norm on the *data space* (`L2NormSquared`), and ∇ is a gradient operator (`odl.Gradient`) in the *reconstruction space*. On discretized images, the gradient operator is approximated by a finite difference scheme.

¹ <https://odl.readthedocs.io>

The idea of total variation and Huber regularization is to force adjacent pixels to have similar values by penalizing gradients. Specifically, the use of ℓ_1 or Huber norm promotes solutions where the gradients are mostly 0 except for a few large jumps. In other words, this promotes cartoon-like images, with regions of near-constant values delineated by sharp borders. The choice of λ in (4) amounts to a trade-off between trying to find some x that fits the data, i.e., $\|A(x) - y\|$ is small, without too much noisy variability, i.e., $H_\epsilon(\nabla x)$ is small.

Note that an attempt to solve (3) directly leads to noisy pixel values (because of ill-posedness). While we saw in the labs that early stopping can still yield reasonable solutions, a suitable choice of the regularization parameter λ in (4) ensures that the issue of semi-convergence is (at least partially) taken care of by the regularization.

Dataset and problem parameters

The 3D CT dataset represents a realistic scan of a human head, obtained with the acquisition geometry of a real CT machine. More instructions on how to access the data and a server with GPU resources will be provided on the course page. For testing purposes in 2D (and 3D), use the Shepp-Logan phantom (`shepp_logan`), noting that the contrast of the unmodified phantom (`modified=False`) is close to the true contrast of the CT dataset. While testing reconstruction methods in 2D, try to obtain enough contrast to distinguish at least some of the Shepp-Logan ellipsoids.

The 3D image to be reconstructed should have resolution $128 \times 128 \times 128$, with `min_pt` and `max_pt` set to `[-112.0, -112.0, 0.0]` and `[112.0, 112.0, 224.0]`. **You must pass `dtype='float32'` when creating the space**, since the ASTRA backend used by ODL 1.0 only accepts single precision; with the default `float64` the ray transform operator will fail with the error message `Failed to link dlpack array: Data must be float32`.

To model the acquisition geometry, import `odl.contrib.tomo` and call `elekta_icon_geometry(num_angles=128)`. Note that `odl.contrib` is not imported automatically by `import odl` and requires an explicit import. Combine this geometry with the reconstruction space to form the ray transform operator via `RayTransform`. The 3D CT data should be in the range space of this operator.

To test reconstruction methods in 2D, define the space as a slice of the 3D space, i.e., with resolution 128×128 and `min_pt` resp. `max_pt` set to `[-112.0, -112.0]` resp. `[112.0, 112.0]`. A suitable geometry is then `cone_beam_geometry` with `src_radius` set to 780.0, `det_radius` set to 220.0, `num_angles` set to 128 and `short_scan` set to `True`.

Tasks

- First write a script for solving the regularized problem in (4). You can use any solver implemented in ODL (`odl.solvers.smooth`), or implement your own. Note that functionals can be composed with operators using `FunctionalComp` (equivalently, `functional * operator`), and translated by a vector using `FunctionalTranslation` (equivalently, `functional.translated(vector)`). For additional information and tips about operator and functional arithmetic in ODL, see the guides: https://odl.readthedocs.io/guide/operator_guide.html, https://odl.readthedocs.io/guide/functional_guide.html.
- Test your script in 2D: try to reconstruct 2D images based on simulated noisy data using 2D phantoms (e.g., `odl.phantom.shepp_logan` with `modified=False`). For reference, the additive noise/corruption in the 3D CT data has a relative norm of about 1/100 (this ratio would otherwise have to be estimated from the data). If you implemented an algorithmic solver yourself, verify that it returns a result similar to that obtained with `steepest_descent`.
- Based on your tests with the Shepp-Logan phantom in 2D, choose a parameter ϵ for the Huber norm and motivate your choice.
- For the regularization parameter λ , you should try to find a value that yields a good compromise between data-fit and regularization. Implement one of the methods from the lectures to find good regularization parameter values. Plot the 2D reconstruction error against the number of solver iterations for a few reasonable values of λ and noise levels.
- ONLY when the script is working for the 2D problem (and reasonable parameter values have been found) should you run it with the 3D CT data. This is important, since the running time for a solver can be very long, depending on the number of iterations. Start with a few iterations. Describe the effect of using different values for λ , and describe qualitatively the evolution of the reconstructed images as the number of iterations increases. Show some 2D-slices of the reconstructions.
- Compute a reconstruction using Filtered Back-Projection (`fbp_op`). Vary the `filter_type` and `frequency_scaling`. In particular, you should try and compare the Ram-Lak and Hann (Hanning) filter types. Frequency scaling is the relative frequency cut-off for the filter, so a smoother image is obtained by choosing smaller values. Compare the results to those obtained in the previous tasks.
- Document your code and results in a written report.

Hints and code examples

- The following imports will be necessary:

```
import numpy as np
import matplotlib.pyplot as plt
import odl
from odl.applications import tomo
from odl.contrib.tomo import elekta_icon_geometry
```

- The following code serves as a starting point for 3D CT:

```
# dtype='float32' is required by ASTRA.
reco_space = odl.uniform_discr(
    [-112, -112, 0],
    [112, 112, 224],
    (128, 128, 128),
    dtype='float32',
)
geometry = elekta_icon_geometry(num_angles=128)
ray_trafo = tomo.RayTransform(
    vol_space=reco_space,
    geometry=geometry,
)
data_space = ray_trafo.range
```

- The following code serves as a starting point for 2D tests:

```
reco_space_2d = odl.uniform_discr(
    [-112, -112],
    [112, 112],
    (128, 128),
    dtype='float32',
)
geometry_2d = tomo.cone_beam_geometry(
    space=reco_space_2d,
    src_radius=780.0,
    det_radius=220.0,
    num_angles=128,
    short_scan=True,
)
ray_trafo_2d = tomo.RayTransform(
    vol_space=reco_space_2d,
    geometry=geometry_2d,
)
data_space_2d = ray_trafo_2d.range
```

- The functionals you need are in the `odl.functionals` namespace, e.g.,

```
l2 = odl.functionals.L2NormSquared(data_space)
data_fit = l2.translated(data)
```

- The simplest way to minimize the objective function $Q(x)$ in (4) is to employ the steepest descent (gradient descent) method:

```

1:  $x_0$  arbitrary,  $r_0 := -\nabla Q(x_0)$ ,  $t$  - step size.
2: for  $k := 0, 1, \dots$  do
3:    $x_{k+1} := x_k + tr_k$ 
4:    $r_{k+1} := -\nabla Q(x_{k+1})$ 
5: end for

```

- The above method may be improved by implementing a line search:

```

1:  $x_0$  arbitrary,  $r_0 := -\nabla Q(x_0)$ ,  $t$  - step size.
2: for  $k := 0, 1, \dots$  do
3:    $x_{k+1} := \text{argmin of } Q \text{ on the half-line } t \mapsto x_k + tr_k, \ t \geq 0$ 
4:    $r_{k+1} := -\nabla Q(x_{k+1})$ 
5: end for

```

ODL provides `BacktrackingLineSearch`, which can be passed as the `line_search` argument of `odl.solvers.steepest_descent` in place of a constant step size.

- Steepest descent converges if the step size t satisfies $t < 2/L$, where L is a Lipschitz constant of ∇Q . For (4) such an L can be assembled from the norms of the operators involved; the Huber term contributes a factor $1/\epsilon$, so a small smoothing parameter forces a small step size.
- The operator ∇Q for a functional Q can be obtained with `Q.gradient`.
- An operator norm can be estimated with `.norm(estimate=True)`. If you have a composed operator you might want to estimate the norms of its components (https://en.wikipedia.org/wiki/Operator_norm).
- `.norm(estimate=True)` returns a NumPy scalar, and ODL 1.0 does not allow a NumPy scalar to multiply a space element from the left: `np.float64(...) * x` raises a `RuntimeError`: wrap such scalars in `float(...)`. For the same reason, use ODL's array-API functions, e.g., `odl.max`, `odl.abs`, etc, rather than the NumPy equivalents.
- Structure your functions so that they can accept general input, i.e., any method should work for both 2D and 3D spaces without issue.
- If your code is slow, try to avoid making copies of intermediate results.
- For more information about ODL, read the user's guide at <https://odl.readthedocs.io/guide/guide.html>.